Generative AI Assignment 3: Questions 1 and 2

English-to-Urdu Machine Translation and Vision Transformer for CIFAR-10 Classification

Muhammad Atlas Malik
Department of Data Science and AI, FAST-NUCES
Islamabad, Pakistan
i248020@isb.nu.edu.pk

***Abstract*—This report presents the implementation and evaluation of two tasks from Assignment 3. Question 1 implements English-to-Urdu machine translation using the UMC005 parallel corpus. A Transformer encoder-decoder was implemented from scratch in PyTorch and compared with an LSTM attention baseline, while a pretrained Helsinki-NLP English-to-Urdu model was also evaluated and fine-tuned as a bonus experiment. The custom Transformer achieved BLEU 5.0330, ROUGE-L 0.2811, and perplexity 81.7818, while the LSTM achieved BLEU 5.3885 and ROUGE-L 0.3243. The fine-tuned pretrained model achieved the strongest machine translation result with BLEU 17.0037, ROUGE-1 0.5175, ROUGE-2 0.2461, and ROUGE-L 0.4643. Question 2 implements image classification on CIFAR-10 using a Vision Transformer trained from scratch and compares it with a Hybrid CNN + MLP model and an ImageNet-pretrained ResNet18 transfer learning model. The ViT achieved test accuracy 0.7536 and F1-score 0.7513. The Hybrid CNN + MLP achieved the strongest CIFAR-10 result with test accuracy 0.8992 and F1-score 0.8997, while ResNet18 achieved test accuracy 0.8598 and F1-score 0.8595. Overall, the results show that pretrained translation knowledge substantially improves English-to-Urdu translation, and convolutional inductive bias remains highly effective for small 32 by 32 image classification.**

***Index Terms*—Transformer, machine translation, English-to-Urdu, UMC005, SentencePiece, BLEU, ROUGE, Vision Transformer, CIFAR-10, image classification, PyTorch, ResNet18**

I. Question 1: Transformer Implementation for Machine Translation

A. Introduction

Question 1 required the implementation of a Transformer model for English-to-Urdu machine translation using the UMC005 English-Urdu Parallel Corpus. The task focused on preprocessing aligned English and Urdu text, training suitable tokenizers, implementing an encoder-decoder Transformer from scratch, evaluating translation quality using BLEU and ROUGE, comparing against an LSTM baseline, visualizing attention, and deploying a local translation interface.

Machine translation is a sequence-to-sequence problem in which a source sentence is mapped into a target sentence while preserving meaning, word order, and grammatical structure. English-to-Urdu translation is especially challenging because the two languages differ in script, morphology, and sentence structure. This work therefore uses subword tokenization and neural encoder-decoder models to reduce vocabulary sparsity and improve generalization. The primary model is a Transformer implemented from scratch in PyTorch, following the self-attention architecture introduced by Vaswani et al. [3]. A bidirectional LSTM encoder-decoder with additive attention was implemented as a comparative baseline [8]. A pretrained Helsinki-NLP English-to-Urdu model was also evaluated and fine-tuned as a bonus experiment [11].

B. Methodology

1) Dataset and Preprocessing: The UMC005 English-Urdu corpus was used through its prepared Bible and Quran domain splits. The train, development, and test sets were built by combining the corresponding split files from the two domains. Larger unsplit files were not used, preventing train-test leakage. The preprocessing pipeline normalized Unicode text, collapsed repeated whitespace, stripped leading and trailing spaces, preserved Urdu script, removed empty aligned pairs, and retained line-level English-Urdu alignment.

TABLE I
Question 1 Dataset Processing Summary

Item	Description
Corpus	UMC005 English-Urdu
Domains	Bible and Quran
Splits	Train, dev, test
Cleaning	Unicode normalization and whitespace cleanup
Alignment	Line-level English-Urdu pairs preserved
Test size	457 sentence pairs

2) Tokenization: Separate SentencePiece BPE tokenizers were trained for English and Urdu [7]. The tokenizer vocabulary size was set to 8000 for each language, with automatic fallback to smaller sizes if the corpus could not support the requested vocabulary. Four special tokens were used: PAD, UNK, BOS, and EOS with IDs 0, 1, 2, and 3 respectively. Each source sentence was encoded as BOS + English pieces + EOS, while each target sentence was shifted into decoder input and decoder output sequences for teacher-forced training.

3) Transformer Architecture: The Transformer was implemented manually in PyTorch rather than using a prebuilt sequence-to-sequence module. The model contained source and target embeddings, sinusoidal positional encoding, multi-head scaled dot-product attention, encoder self-attention, decoder masked self-attention, decoder cross-attention, feed-forward layers, residual connections, layer normalization, dropout, source padding masks, and target causal masks.

The encoder maps English source token IDs into contextual hidden states. The decoder receives shifted Urdu target tokens

and generates Urdu outputs autoregressively. During training, causal masking prevents the decoder from attending to future target positions. During inference, greedy decoding is used until EOS or the maximum sequence length is reached.

TABLE II
QUESTION 1 TRANSFORMER CONFIGURATION

Hyperparameter	Value
Layers	3
d_{model}	256
Attention heads	4
Feed-forward size	1024
Dropout	0.1
Maximum sequence length	80
Vocabulary size	8000 per language

The attention operation was computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \qquad (1)$$

where Q, K, and V represent query, key, and value matrices, and d_k is the key dimension.

4) LSTM Baseline: For comparison, an LSTM encoder-decoder baseline with attention was implemented. The encoder used a bidirectional LSTM, while the decoder used an LSTM with additive attention over encoder states. The LSTM used teacher forcing during training and greedy decoding during inference. This baseline was included because recurrent sequence-to-sequence models remain useful for smaller datasets and provide a meaningful comparison against the from-scratch Transformer.

TABLE III
QUESTION 1 LSTM BASELINE CONFIGURATION

Hyperparameter	Value
Embedding size	256
Hidden size	256
Layers	2
Dropout	0.2
Attention	Additive attention
Maximum sequence length	80
Teacher forcing ratio	0.5

5) Training Setup: Both custom models were trained using cross-entropy loss with padding tokens ignored. AdamW optimization, ReduceLROnPlateau scheduling, gradient clipping, validation monitoring, checkpoint saving, and early stopping were used. Mixed precision AMP was enabled for Transformer training when CUDA was available. The default training setup used batch size 32, learning rate 3×10^{-4}, 20 maximum epochs, patience 4, weight decay 1×10^{-4}, gradient clipping 1.0, and random seed 42.

6) Evaluation and Deployment: Evaluation was performed on the 457-sentence combined test set. BLEU was computed using SacreBLEU [9], while ROUGE-1, ROUGE-2, and ROUGE-L were computed using token overlap and longest common subsequence matching [10]. Perplexity was computed from test cross-entropy. The implementation also measured total inference time, average inference time, training time, and peak GPU memory. A Streamlit GUI was implemented for local deployment, allowing the user to select a model, enter an English sentence, and view the Urdu translation in a chat-like interface with Urdu text right-aligned.

TABLE IV
QUESTION 1 SHARED TRAINING CONFIGURATION

Setting	Value
Framework	PyTorch
Batch size	32
Learning rate	$3e^{-4}$
Epochs	20
Patience	4
Weight decay	$1e^{-4}$
Gradient clipping	1.0
Loss	Cross entropy

C. Results

1) Training Curves: The Transformer loss curve in Fig. 1 shows rapid improvement during the early epochs. The training loss continued to decrease, while validation loss flattened after the middle of training. The Transformer reached its best validation loss at epoch 13 with validation loss 3.9369 and early-stopped at epoch 17.

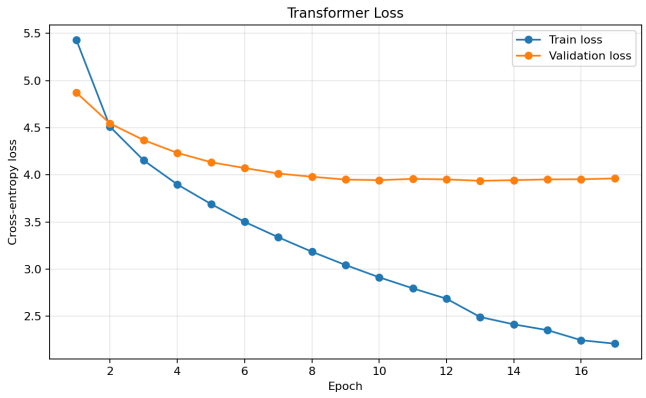

Fig. 1. Transformer training and validation loss curve for English-to-Urdu translation.

The LSTM loss curve in Fig. 2 shows a smoother decrease in training loss across all 20 epochs. Its validation loss improved more slowly and reached its best value at epoch 20 with validation loss 4.1894.

2) Custom Transformer and LSTM Evaluation: Table V summarizes the test results for all Question 1 models on the 457-sentence test set. The LSTM achieved slightly higher BLEU and ROUGE scores than the custom Transformer, while the Transformer achieved better perplexity and much shorter training time. The pretrained Helsinki-NLP model outperformed both custom models, and the fine-tuned pretrained model achieved the strongest overall translation quality.

3) Bonus Pretrained Experiment: The bonus experiment evaluated the pretrained `Helsinki-NLP/opus-mt-en-ur` model and then fine-tuned it for one epoch using 1,000 training pairs

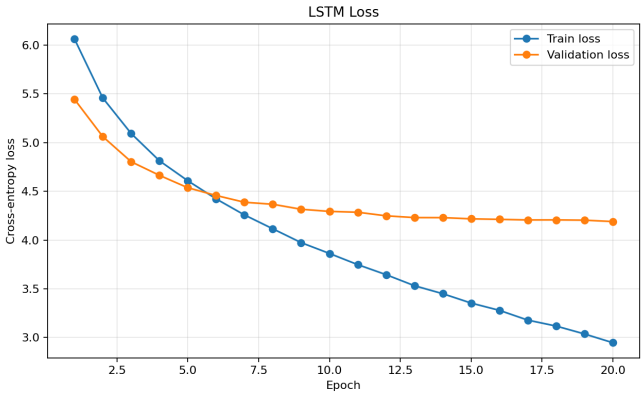

Fig. 2. LSTM training and validation loss curve for English-to-Urdu translation.

TABLE V
QUESTION 1 TEST TRANSLATION METRICS

Model	BLEU	R-1	R-2	R-L	PPL
Transformer	5.033	0.3414	0.0925	0.2811	81.7818
LSTM	5.3885	0.3699	0.102	0.3243	103.7877
Pretrained	11.2855	0.4335	0.1888	0.3875	–
Fine-tuned	17.0037	0.5175	0.2461	0.4643	–

and 200 development pairs. As shown in Table V, the fine-tuned pretrained model achieved the strongest overall automatic metrics, reaching BLEU 17.0037, ROUGE-1 0.5175, ROUGE-2 0.2461, and ROUGE-L 0.4643 on the same 457-sentence test set.

4) Attention Visualization: The Transformer attention visualization script extracted decoder cross-attention weights from generated translations and saved heatmaps. Fig. 3 and Fig. 4 show two examples. The x-axis represents English source subword tokens and the y-axis represents generated Urdu subword tokens. Brighter cells indicate stronger attention weights. The heatmaps show that the decoder often concentrates on selected source positions while generating Urdu tokens, although attention remains noisy because the custom Transformer was trained on a relatively small domain-specific corpus.

Sample qualitative outputs confirm the same pattern. For the input sentence "And to them it was given that they should not kill them," the model generated an Urdu sentence with related concepts such as killing and punishment, but it did not fully preserve the exact reference structure. For the sentence about seeking death and not finding it, the generated output captured repeated references to death but produced repetitive phrasing.

TABLE VI
QUESTION 1 EFFICIENCY COMPARISON

Model	Train s	GPU MB	Avg Inf. s	Total Inf. s
Transformer	363.381	358.49	0.177056	80.9145
LSTM	3729.934	490.30	0.035082	16.0326
Pretrained	–	333.42	0.335657	153.3951
Fine-tuned	–	330.60	0.279619	127.786

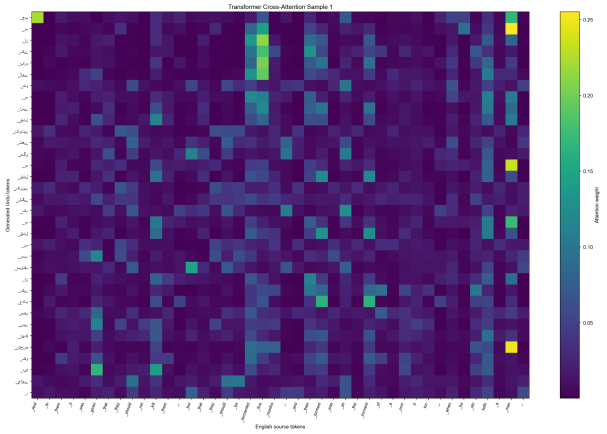

Fig. 3. Transformer cross-attention heatmap for translation sample 1.

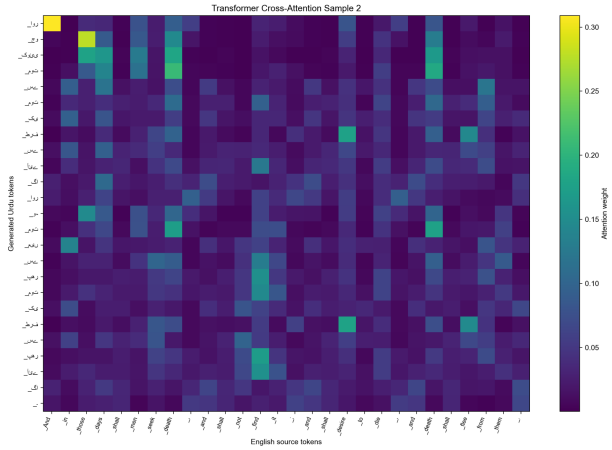

Fig. 4. Transformer cross-attention heatmap for translation sample 2.

This shows that the model learned domain vocabulary but still struggled with long-range semantic fidelity.

D. Discussion

The evaluation shows a clear tradeoff between the custom Transformer and LSTM baseline. The Transformer trained in 363.381 seconds compared with 3729.934 seconds for the LSTM, making it far more efficient during training. It also achieved lower perplexity, suggesting stronger token-level predictive confidence. However, the LSTM achieved slightly better BLEU and ROUGE scores on the test set. This is plausible because the corpus is small, religious, and formulaic. On such a dataset, recurrent models with attention can remain competitive, especially when longer sequential training allows them to memorize common phrase structures.

The pretrained model results show the strongest performance. The unfine-tuned pretrained model achieved BLEU 11.2855, ROUGE-1 0.4335, ROUGE-2 0.1888, and ROUGE-L 0.3875, already outperforming the custom Transformer and LSTM. After one epoch of in-domain fine-tuning, BLEU increased to 17.0037, ROUGE-1 increased to 0.5175, ROUGE-2 increased to 0.2461, and ROUGE-L increased to 0.4643.

The fine-tuned model used 330.60 MB peak GPU memory and averaged 0.279619 seconds per sentence during inference. This confirms that pretrained multilingual or translation-specific knowledge provides a major advantage when parallel data are limited.

The main challenge was the limited and domain-specific nature of the corpus. Many sentences are religious and formulaic, so the model can learn repeated vocabulary but may not generalize well to everyday English. Urdu tokenization is also difficult because of rich morphology and script variation. Greedy decoding was simple and reproducible, but beam search would likely improve translation fluency. Finally, BLEU and ROUGE scores remain modest because exact lexical overlap is difficult for Urdu translation, where multiple valid translations can differ significantly from the reference.

E. Conclusion

Question 1 implemented a complete English-to-Urdu translation workflow using the UMC005 corpus. The system included split-safe preprocessing, SentencePiece BPE tokenization, a from-scratch Transformer, an LSTM attention baseline, training logs, BLEU/ROUGE/perplexity evaluation, attention heatmaps, and a local Streamlit deployment. The custom Transformer achieved BLEU 5.033, ROUGE-1 0.3414, ROUGE-2 0.0925, ROUGE-L 0.2811, and perplexity 81.7818, while the LSTM achieved BLEU 5.3885, ROUGE-1 0.3699, ROUGE-2 0.102, ROUGE-L 0.3243, and perplexity 103.7877. The bonus fine-tuned pretrained model achieved the best result with BLEU 17.0037, ROUGE-1 0.5175, ROUGE-2 0.2461, and ROUGE-L 0.4643. Overall, the custom Transformer successfully demonstrated the required architecture, but pretrained translation knowledge produced the strongest translation quality.

II. Question 2: Vision Transformer for CIFAR-10 Image Classification

A. Introduction

Image classification is a core computer vision task where a model assigns an input image to one of several semantic classes. Question 2 of the assignment required implementing a Vision Transformer (ViT) for CIFAR-10 image classification, tuning important hyperparameters, evaluating the model using classification metrics, comparing it with traditional or hybrid architectures, and visualizing both training behavior and prediction results.

CIFAR-10 is a ten-class dataset containing 60,000 color images at 32 by 32 resolution, divided into 50,000 training images and 10,000 test images [1]. The ten classes are airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Compared with large-scale image datasets, CIFAR-10 is small and low resolution. This makes it a useful test case for comparing Transformer-based image models with convolutional models, because Transformers usually need more data to learn spatial structure from scratch.

The primary model in this work is a compact ViT inspired by Dosovitskiy et al. [2]. Instead of applying convolutions across the full image, the ViT divides each image into non-overlapping patches, converts those patches into tokens, adds positional embeddings, and processes the token sequence using multi-head self-attention. The attention mechanism follows the Transformer formulation introduced by Vaswani et al. [3]. For comparison, two baselines were implemented: a Hybrid CNN + MLP model and a ResNet18 transfer learning model [4]. The comparison was based on accuracy, precision, recall, F1-score, training time, GPU memory, inference speed, and qualitative visualization of correct and incorrect predictions.

B. Methodology

1) Dataset and Preprocessing: The local CIFAR-10 Python dataset was used through `torchvision.datasets.CIFAR10` with `download=False` [6]. The original training split of 50,000 images was divided into 45,000 training images and 5,000 validation images using a fixed random seed of 42. The official test split of 10,000 images was used for final evaluation. Fig. 5 shows sample CIFAR-10 images used in the project.

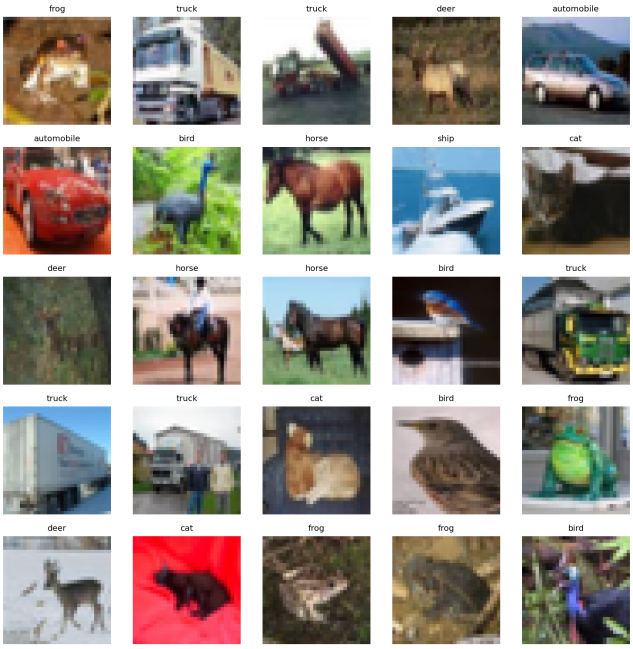

Fig. 5. Sample images from the CIFAR-10 dataset.

For the ViT and Hybrid CNN + MLP models, images were kept at 32 by 32 resolution. Training augmentation used random cropping with padding of 4 pixels and random horizontal flipping. Images were then converted to tensors and normalized using CIFAR-10 channel statistics: mean $(0.4914, 0.4822, 0.4465)$ and standard deviation $(0.2470, 0.2435, 0.2616)$. Validation and test data used only tensor conversion and normalization. Fig. 6 shows augmentation examples.

For ResNet18 transfer learning, images were resized to 224 by 224 pixels because ImageNet-pretrained ResNet18

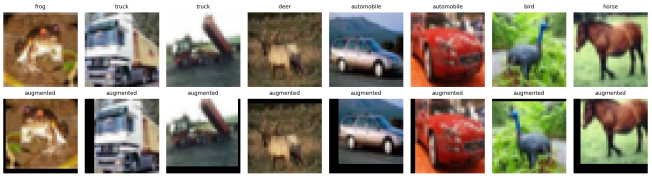

Fig. 6. Examples of random crop and horizontal flip augmentation used during training.

expects larger ImageNet-style inputs. The ResNet pipeline used ImageNet normalization with mean $(0.485, 0.456, 0.406)$ and standard deviation $(0.229, 0.224, 0.225)$.

TABLE VII
DATASET SPLIT AND PREPROCESSING SUMMARY

Stage	Images	Transformations
Training	45,000	Crop, flip, normalize
Validation	5,000	Normalize only
Test	10,000	Normalize only

2) Vision Transformer Architecture: The ViT was implemented from scratch in PyTorch rather than using a prebuilt Transformer classifier. A 32 by 32 input image was divided into 4 by 4 non-overlapping patches. This produced an 8 by 8 grid, or 64 patch tokens per image. Patch embedding was implemented using a `Conv2d` layer whose kernel size and stride were both equal to the patch size. This is equivalent to cutting the image into patches and projecting each patch into a learnable embedding space.

A learnable class token was prepended to the patch sequence. Learnable positional embeddings were added so the model could distinguish patch locations. The token sequence was then passed through six Transformer encoder blocks. Each block used LayerNorm, custom multi-head self-attention, residual connections, and a feed-forward MLP with GELU activation. The final class token representation was passed to a linear classifier for ten-class prediction.

TABLE VIII
VIT ARCHITECTURE CONFIGURATION

Component	Value
Input size	$32 \times 32 \times 3$
Patch size	4×4
Number of patches	64
Embedding dimension	192
Transformer depth	6 blocks
Attention heads	6
MLP hidden dimension	384
Dropout	0.1
Parameters	1,212,490

The attention computation followed the standard scaled dot-product formulation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \qquad (2)$$

where Q, K, and V are query, key, and value projections, and d_k is the per-head key dimension.

3) Hybrid CNN + MLP Baseline: The first comparison model was a Hybrid CNN + MLP classifier. It used three convolutional blocks with Batch Normalization and ReLU activation. Max pooling reduced the spatial size after the first two convolutional blocks, and adaptive average pooling reduced the final feature map to 2 by 2. The flattened feature vector was classified using a three-layer MLP. This model intentionally avoided self-attention and Transformer blocks, making it a clean convolutional baseline.

The Hybrid CNN + MLP had 1,160,138 trainable parameters. Its purpose was to test whether a smaller model with strong local-feature inductive bias would outperform a from-scratch ViT on CIFAR-10.

4) ResNet18 Transfer Learning Baseline: The second comparison model used ResNet18 pretrained on ImageNet. The final fully connected layer was replaced with a ten-class CIFAR-10 classifier. Training was performed in two stages. First, the backbone was frozen and only the classification head was trained for 10 epochs. Second, the final residual stage, `layer4`, and the classification head were unfrozen and fine-tuned for 5 additional epochs. This produced a total of 15 training epochs.

The ResNet18 model contained 11,181,642 parameters. Although it had access to pretrained features, it also required resizing CIFAR-10 images to 224 by 224, making inference slower and GPU memory usage higher than the smaller 32 by 32 models.

5) Training Configuration: All models were trained using PyTorch [5]. The ViT used AdamW with weight decay and cosine annealing, which is commonly used for Transformer training. The Hybrid CNN + MLP used Adam with ReduceLROnPlateau scheduling. The ResNet18 model used Adam during head training and a lower learning rate during fine-tuning. Cross-entropy loss was used for all three classifiers.

TABLE IX
TRAINING CONFIGURATION FOR ALL MODELS

Config.	ViT	CNN+MLP	ResNet18
Framework	PyTorch	PyTorch	PyTorch
Input	32×32	32×32	224×224
Batch	128	128	64
Epochs	30	25	10+5
Optimizer	AdamW	Adam	Adam
LR	$3e^{-4}$	$1e^{-3}$	$1e^{-3}$, $1e^{-4}$
Scheduler	Cosine	Plateau	Stage-wise
Loss	CE	CE	CE
Aug.	Crop+flip	Crop+flip	Resize+flip

6) Hyperparameter Tuning: A lightweight ViT hyperparameter search was performed before the final run. The search compared patch size, depth, number of heads, and learning rate using small subsets to keep tuning fast. The configuration with patch size 4, depth 4, 4 heads, and learning rate 3×10^{-4} gave the highest quick tuning validation accuracy. However, the final model used the deeper 6-block, 6-head configuration because it had greater representational capacity for the full training run and matched the intended assignment objective of implementing a multi-layer Transformer.

TABLE X
LIGHTWEIGHT VIT HYPERPARAMETER TUNING RESULTS

Patch	Depth	Heads	LR	Val. Acc.
4	4	4	$3e^{-4}$	0.2832
4	6	6	$3e^{-4}$	0.2734
8	6	6	$3e^{-4}$	0.2734
4	6	6	$1e^{-4}$	0.2637

C. Results

1) Vision Transformer Results: The ViT showed steady convergence over 30 epochs. Training accuracy increased from 31.70% in epoch 1 to 75.88% in epoch 30. Validation accuracy increased from 40.78% to 76.38%. Validation loss dropped from 1.6478 to 0.6912, showing that the model learned meaningful image representations from scratch. The final test accuracy was 0.7536 (75.36%), with weighted precision of 0.7530, recall of 0.7536, and weighted F1-score of 0.7513.

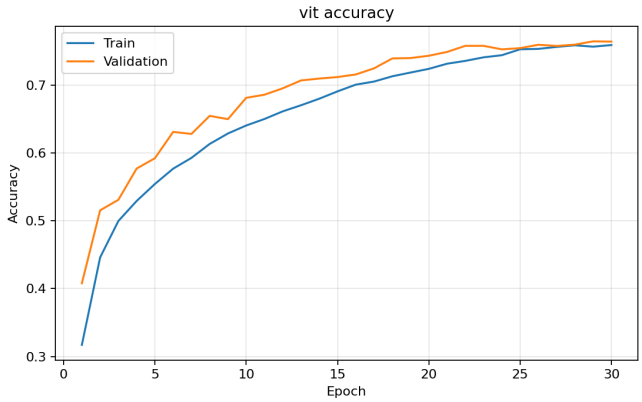

Fig. 7. ViT training and validation accuracy curves.

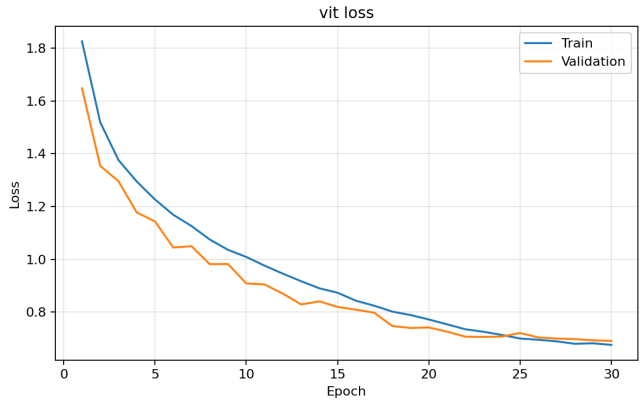

Fig. 8. ViT training and validation loss curves.

The class-level results show that the ViT performed well on automobile, ship, frog, and truck classes. The weakest classes were cat and dog, where confusion is common because both classes contain animals with similar textures, backgrounds, and shapes at 32 by 32 resolution. The confusion matrix in Fig. 9 confirms this pattern.

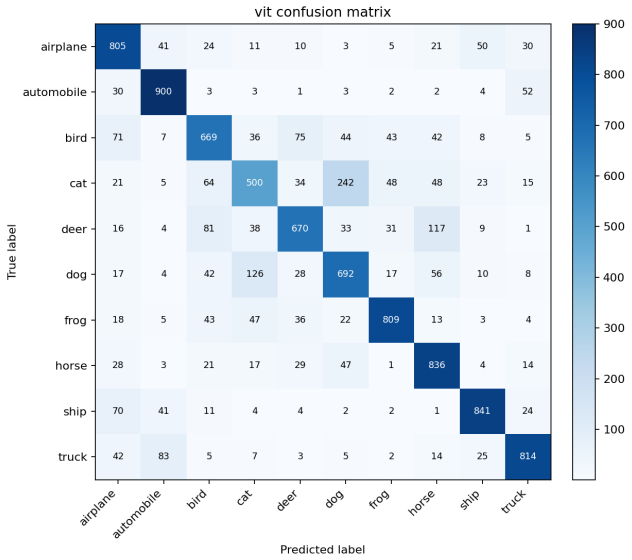

Fig. 9. Confusion matrix for the ViT model on the CIFAR-10 test set.

2) Hybrid CNN + MLP Results: The Hybrid CNN + MLP was the best-performing model in this work. It reached a best validation accuracy of 0.9040 and final test accuracy of 0.8992 (89.92%). Its weighted F1-score was 0.8997, with weighted precision of 0.9014 and recall of 0.8992. Training accuracy increased from 43.88% to 95.26%, while validation accuracy rose from 52.66% to 90.28% by the final epoch.

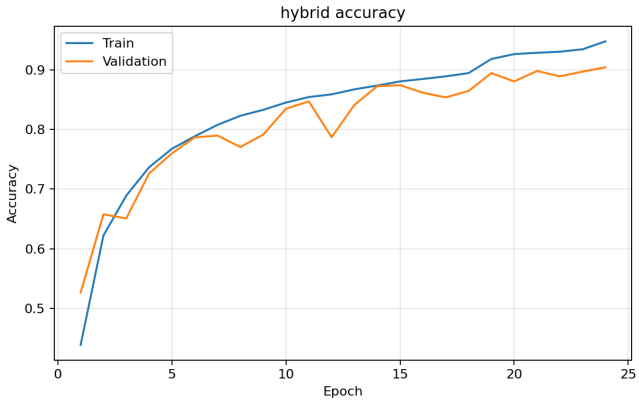

Fig. 10. Hybrid CNN + MLP training and validation accuracy curves.

The model performed especially well on automobile, frog, horse, ship, and truck classes. Cat remained the most difficult class, but even there the Hybrid CNN + MLP achieved much stronger results than the ViT. Fig. 12 shows the model's confusion matrix.

3) ResNet18 Transfer Learning Results: The ResNet18 transfer learning model achieved a best validation accuracy of 0.8658 and a test accuracy of 0.8598 (85.98%). Its weighted

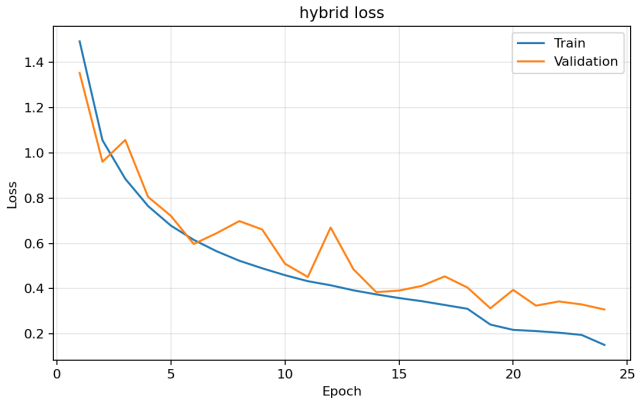

Fig. 11. Hybrid CNN + MLP training and validation loss curves.

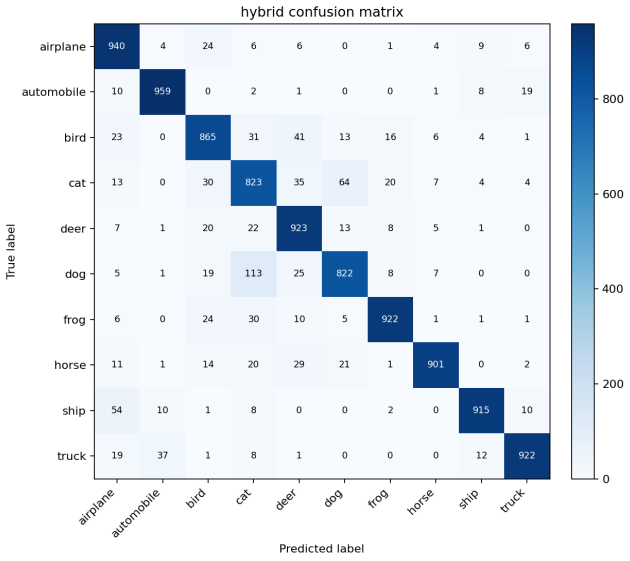

Fig. 12. Confusion matrix for the Hybrid CNN + MLP model.

precision, recall, and F1-score were 0.8599, 0.8598, and 0.8595, respectively. During the frozen-backbone stage, validation accuracy reached 82.16%. After unfreezing `layer4`, validation accuracy improved to 86.58%, confirming that final-stage fine-tuning helped adapt ImageNet features to CIFAR-10.

The ResNet18 model gave balanced performance across most classes. Its main weakness was again the cat class, followed by dog, which is consistent with the other two models. The confusion matrix is shown in Fig. 15.

4) Overall Comparison: Table XI compares best validation accuracy and test performance, while Tables XII and XIII compare efficiency and resource usage. The Hybrid CNN + MLP achieved the highest accuracy and F1-score, the fastest inference speed at 3656.9879 images per second, and the shortest total training time at 315.2951 seconds. The ResNet18 model achieved better accuracy than the ViT, but it required the most parameters, the highest peak GPU memory at 1547.7998

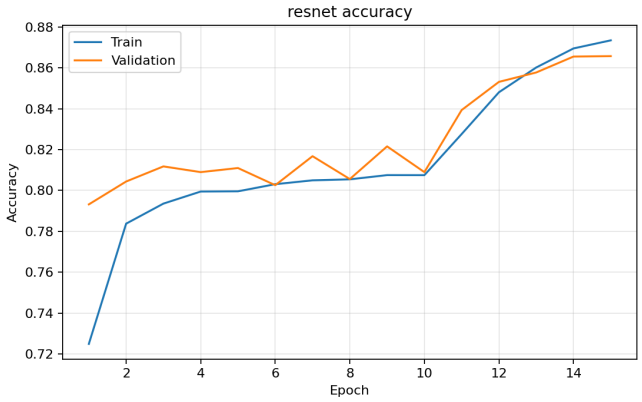

Fig. 13. ResNet18 training and validation accuracy curves.

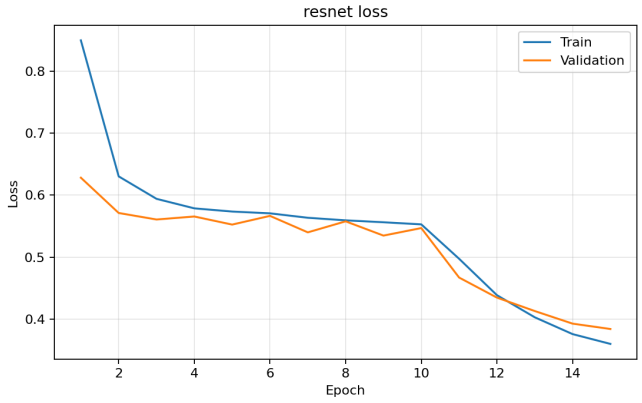

Fig. 14. ResNet18 training and validation loss curves.

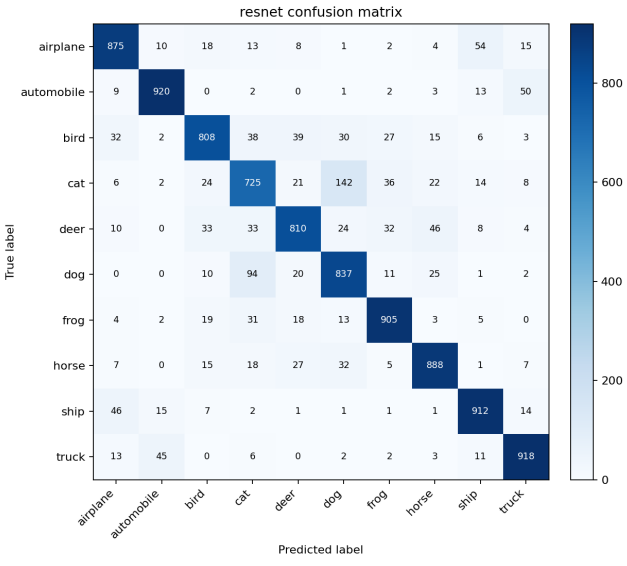

Fig. 15. Confusion matrix for the ResNet18 transfer learning model.

MB, and the slowest inference because its input resolution was resized to 224 by 224. The ViT had the lowest GPU memory usage at 498.6870 MB but also the lowest accuracy.

TABLE XI
MODEL COMPARISON ON CIFAR-10 TEST SET

Model	Best Val.	Test	Prec.	Rec.	F1
ViT scratch	0.7638	0.7536	0.7530	0.7536	0.7513
Hybrid CNN+MLP	0.9040	0.8992	0.9014	0.8992	0.8997
ResNet18 transfer	0.8658	0.8598	0.8599	0.8598	0.8595

TABLE XII
TRAINING AND INFERENCE SPEED

Model	Train Time (s)	Images/s
ViT scratch	410.5695	788.4147
Hybrid CNN+MLP	315.2951	3656.9879
ResNet18 transfer	898.7038	372.4158

TABLE XIII
MODEL SIZE AND GPU MEMORY USAGE

Model	Parameters	Peak GPU MB
ViT scratch	1,212,490	498.6870
Hybrid CNN+MLP	1,160,138	725.2598
ResNet18 transfer	11,181,642	1547.7998

5) Local Deployment and Prediction Visualization: The assignment also required local or cloud deployment with visualization of classification results. A local demo script, `local_demo.py`, was implemented to load a saved checkpoint, run inference on test images, and save a grid of predicted labels. This provided a lightweight local deployment workflow without requiring a web interface. Fig. 16 shows a sample visualization generated from the local demo output.

In addition to the demo grid, separate visualizations of correct and incorrect predictions were generated for each model. These visual outputs helped identify failure cases, especially visually similar categories such as cat/dog, deer/horse, and automobile/truck.

D. Discussion

1) Why the Hybrid Model Performed Best: The strongest result came from the Hybrid CNN + MLP model. This is expected for CIFAR-10 because 32 by 32 images contain strong local patterns such as edges, corners, textures, and small object parts. Convolutional layers are naturally biased toward local spatial structure and translation consistency, so they learn efficiently from limited data. The CNN therefore did not need to rediscover basic image locality from scratch.

The Hybrid CNN + MLP also had nearly the same parameter count as the ViT, but its parameters were arranged more effectively for small images. Its inference speed of 3656.9879 images per second was much higher than both alternatives, making it the most practical model in this experiment.

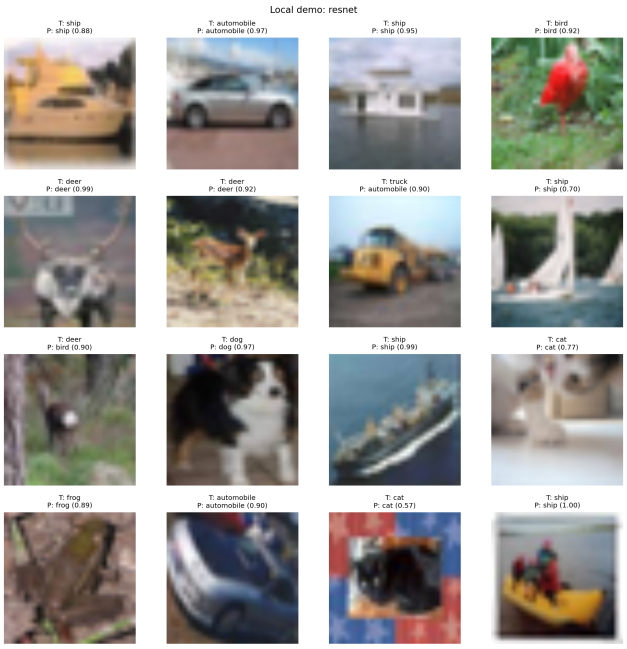

Fig. 16. Local demo prediction grid generated from the saved model checkpoint.

2) ViT Strengths and Weaknesses: The ViT successfully learned from scratch and reached 0.7536 test accuracy, which confirms that the implementation worked correctly. Its training and validation curves showed stable learning without collapse. The model also used the lowest GPU memory among the three architectures.

However, the ViT underperformed the CNN-based models. This is mainly because Transformers have weaker built-in spatial assumptions than CNNs. A ViT must learn relationships between patches from the data itself, while CNNs already encode locality through convolution. On larger datasets, ViTs can become highly competitive or superior, but on CIFAR-10 from scratch, the data size and image resolution limit their advantage.

3) ResNet18 Transfer Learning Behavior: The ResNet18 model benefited from ImageNet-pretrained features and outperformed the ViT by more than 10 percentage points in test accuracy. Fine-tuning `layer4` improved validation accuracy, proving that the pretrained representation still needed adaptation for CIFAR-10. The downside was computational cost. Resizing each 32 by 32 CIFAR image to 224 by 224 increased memory usage and reduced inference speed.

4) Common Classification Errors: Across all models, animal classes were harder than vehicle classes. Cat and dog were consistently the weakest classes because CIFAR-10 images are low resolution, often cluttered, and visually ambiguous. Vehicle classes such as automobile, ship, and truck were easier because their shapes and backgrounds tend to be more distinct.

The incorrect prediction grids support this conclusion: many failures occurred when the object was small, partially occluded, or surrounded by a confusing background.

5) Challenges Encountered: The first challenge was balancing model capacity and training time. A larger ViT could improve accuracy, but it would also increase memory usage and overfitting risk. The second challenge was fair comparison: the ResNet used pretrained ImageNet features while the ViT and Hybrid CNN + MLP were trained from scratch. This is why the report treats ResNet18 as a transfer learning baseline rather than a direct architectural equality test. The third challenge was that CIFAR-10's low resolution made fine-grained categories difficult, especially cat, dog, deer, and horse.

E. Conclusion

This work implemented Question 2 of Assignment 3 by training a Vision Transformer from scratch on CIFAR-10 and comparing it with a Hybrid CNN + MLP model and a pretrained ResNet18 transfer learning model. The ViT achieved 0.7536 test accuracy, showing that the Transformer pipeline, patch embedding, positional encoding, and multi-head self-attention were implemented successfully. However, the Hybrid CNN + MLP achieved the best performance with 0.8992 test accuracy and 0.8997 weighted F1-score. ResNet18 transfer learning achieved 0.8598 test accuracy, outperforming the ViT but requiring more memory and slower inference.

The results demonstrate that for small, low-resolution datasets such as CIFAR-10, convolutional inductive bias remains extremely powerful. A from-scratch ViT can learn useful global patch relationships, but it generally requires more data, stronger augmentation, or pretraining to compete with CNN-based models. Future improvements could include longer ViT training, stronger regularization, DeiT-style distillation, MixUp or CutMix augmentation, larger embedding dimensions, and pretrained Transformer backbones.

PROMPTS

- Develop an English-to-Urdu machine translation pipeline using the UMC005 parallel corpus with split-safe preprocessing and line-level alignment preservation.
- Train separate SentencePiece BPE tokenizers for English and Urdu, including PAD, UNK, BOS, and EOS special tokens.
- Implement a Transformer encoder-decoder model from scratch in PyTorch with positional encoding, multi-head attention, masked decoder attention, cross-attention, feed-forward layers, residual connections, layer normalization, and dropout.
- Implement an LSTM encoder-decoder baseline with bidirectional encoding, additive attention, teacher forcing, and greedy decoding.
- Train and evaluate the custom Transformer and LSTM models using cross-entropy loss, BLEU, ROUGE, perplexity, inference time, training time, and GPU memory usage.
- Generate Transformer cross-attention heatmaps to visualize which English source tokens the decoder attends to while producing Urdu tokens.
- Build a Streamlit local deployment interface where English inputs are left-aligned, Urdu outputs are right-aligned, and translation history remains visible.
- Develop a Vision Transformer model for CIFAR-10 image classification using patch embeddings, positional encodings, a class token, multi-head self-attention, and Transformer encoder layers.
- Prepare the CIFAR-10 data pipeline with training, validation, and test splits, including data augmentation, tensor conversion, and normalization.
- Design and implement comparison models for CIFAR-10 classification, including a Hybrid CNN + MLP architecture and a ResNet18 transfer learning model.
- Train the implemented image classification models using suitable optimizers, learning-rate schedules, checkpoint saving, and validation monitoring.
- Evaluate the trained image classification models using test accuracy, precision, recall, F1-score, confusion matrices, model size, GPU memory usage, training time, and inference speed.
- Generate visual outputs for the report, including training curves, validation curves, confusion matrices, prediction samples, and examples of correct and incorrect classifications.
- Organize the implementation details, experimental results, visualizations, and analysis into an IEEE-style report for Assignment 3 Questions 1 and 2.

REFERENCES

[1] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Technical Report, 2009.

[2] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. ICLR*, 2021.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017.

[4] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.

[5] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

[6] S. Marcel and Y. Rodriguez, "Torchvision: The machine-vision package of Torch," in *Proc. ACM Multimedia*, 2010.

[7] T. Kudo and J. Richardson, "SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing," in *Proc. EMNLP: System Demonstrations*, 2018, pp. 66–71.

[8] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. ICLR*, 2015.

[9] K. Papineni, S. Roukos, T. Ward, and W. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. ACL*, 2002, pp. 311–318.

[10] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Proc. Workshop on Text Summarization Branches Out*, 2004, pp. 74–81.

[11] J. Tiedemann and S. Thottingal, "OPUS-MT: Building open translation services for the world," in *Proc. EAMT*, 2020.